

- Specify [MapTileLayer.BackgroundReplacement](#) as the value of the [Layer](#) property of the [MapTileSource](#).
- Specify [MapStyle.None](#) as the value of the [Style](#) property of the [MapControl](#).

Related topics

- [Bing Maps Developer Center](#) 
- [UWP map sample](#) 
- [Design guidelines for maps](#)

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Perform geocoding and reverse geocoding

Article • 06/21/2024

Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#)  and [Bing Maps documentation](#).

Note

[MapControl](#) and map services require a maps authentication key called a [MapServiceToken](#). For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

This guide shows you how to convert street addresses to geographic locations (geocoding) and convert geographic locations to street addresses (reverse geocoding) by calling the methods of the [MapLocationFinder](#) class in the [Windows.Services.Maps](#) namespace.

The classes involved in geocoding and reverse geocoding are organized as follows.

- The [MapLocationFinder](#) class contains methods that handle geocoding ([FindLocationsAsync](#)) and reverse geocoding ([FindLocationsAtAsync](#)).
- These methods both return a [MapLocationFinderResult](#) instance.
- The [Locations](#) property of the [MapLocationFinderResult](#) exposes a collection of [MapLocation](#) objects.
- [MapLocation](#) objects have both an [Address](#) property, which exposes a [MapAddress](#) object representing a street address, and a [Point](#) property, which exposes a [Geopoint](#) object representing a geographic location.

Important

You must specify a maps authentication key before you can use map services. For more info, see [Request a maps authentication key](#).

Get a location (Geocode)

This section shows how to convert a street address or a place name to a geographic location (geocoding).

1. Call one of the overloads of the [FindLocationsAsync](#) method of the [MapLocationFinder](#) class with a place name or street address.
2. The [FindLocationsAsync](#) method returns a [MapLocationFinderResult](#) object.
3. Use the [Locations](#) property of the [MapLocationFinderResult](#) to expose a collection [MapLocation](#) objects. There may be multiple [MapLocation](#) objects because the system may find multiple locations that correspond to the given input.

C#

```
using Windows.Services.Maps;
using Windows.Devices.Geolocation;
...
private async void geocodeButton_Click(object sender, RoutedEventArgs e)
{
    // The address or business to geocode.
    string addressToGeocode = "Microsoft";

    // The nearby location to use as a query hint.
    BasicGeoposition queryHint = new BasicGeoposition();
    queryHint.Latitude = 47.643;
    queryHint.Longitude = -122.131;
    Geopoint hintPoint = new Geopoint(queryHint);

    // Geocode the specified address, using the specified reference point
    // as a query hint. Return no more than 3 results.
    MapLocationFinderResult result =
        await MapLocationFinder.FindLocationsAsync(
            addressToGeocode,
            hintPoint,
            3);

    // If the query returns results, display the coordinates
    // of the first result.
    if (result.Status == MapLocationFinderStatus.Success)
    {
        tbOutputText.Text = "result = (" +
            result.Locations[0].Point.Position.Latitude.ToString() + "," +
            result.Locations[0].Point.Position.Longitude.ToString() + ")";
    }
}
```

This code displays the following results to the `tbOutputText` textbox.

Output

```
result = (47.6406099647284, -122.129339994863)
```

Get an address (reverse geocode)

This section shows how to convert a geographic location to an address (reverse geocoding).

1. Call the `FindLocationsAtAsync` method of the `MapLocationFinder` class.
2. The `FindLocationsAtAsync` method returns a `MapLocationFinderResult` object that contains a collection of matching `MapLocation` objects.
3. Use the `Locations` property of the `MapLocationFinderResult` to expose a collection `MapLocation` objects. There may be multiple `MapLocation` objects because the system may find multiple locations that correspond to the given input.
4. Access `MapAddress` objects through the `Address` property of each `MapLocation`.

C#

```
using Windows.Services.Maps;
using Windows.Devices.Geolocation;
...
private async void reverseGeocodeButton_Click(object sender, RoutedEventArgs e)
{
    // The location to reverse geocode.
    BasicGeoposition location = new BasicGeoposition();
    location.Latitude = 47.643;
    location.Longitude = -122.131;
    Geopoint pointToReverseGeocode = new Geopoint(location);

    // Reverse geocode the specified geographic location.
    MapLocationFinderResult result =
        await
    MapLocationFinder.FindLocationsAtAsync(pointToReverseGeocode);

    // If the query returns results, display the name of the town
    // contained in the address of the first result.
    if (result.Status == MapLocationFinderStatus.Success)
    {
        tbOutputText.Text = "town = " +
            result.Locations[0].Address.Town;
    }
}
```

This code displays the following results to the `tbOutputText` textbox.

```
syntax
```

```
town = Redmond
```

Related topics

- [UWP map sample](#) 
- [Design guidelines for maps](#)
- [Bing Maps Developer Center](#) 
- [MapLocationFinder](#) class
- [FindLocationsAsync](#) method
- [FindLocationsAtAsync](#) method

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Get the user's location

Article • 06/21/2024

❗ Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#) [↗] and [Bing Maps documentation](#).

ⓘ Note

[MapControl](#) and map services require a maps authentication key called a [MapServiceToken](#). For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

Find the user's location and respond to changes in location. Access to the user's location is managed by privacy settings in the Windows Settings app. This topic also shows how to check if your app has permission to access the user's location.

Enable the location capability

1. In **Solution Explorer**, double-click **package.appxmanifest** and select the **Capabilities** tab.
2. In the **Capabilities** list, check the box for **Location**. This adds the `location` device capability to the package manifest file.

XML

```
<Capabilities>
  <!-- DeviceCapability elements must follow Capability elements (if
present) -->
  <DeviceCapability Name="location"/>
</Capabilities>
```

Get the current location

This section describes how to detect the user's geographic location using APIs in the [Windows.Devices.Geolocation](#) namespace.

Step 1: Request access to the user's location

Unless your app has coarse location capability (see note), you must request access to the user's location by using the [RequestAccessAsync](#) method before attempting to access the location. You must call the **RequestAccessAsync** method from the UI thread and your app must be in the foreground. Your app will not be able to access the user's location information until after the user grants permission to your app.*

C#

```
using Windows.Devices.Geolocation;
...
var accessStatus = await Geolocator.RequestAccessAsync();
```

The [RequestAccessAsync](#) method prompts the user for permission to access their location. The user is only prompted once (per app). After the first time they grant or deny permission, this method no longer prompts the user for permission. To help the user change location permissions after they've been prompted, we recommend that you provide a link to the location settings as demonstrated later in this topic.

ⓘ Note

The coarse location feature allows your app to obtain an intentionally obfuscated (imprecise) location without getting the user's explicit permission (the system-wide location switch must still be **on**, however). To learn how to utilize coarse location in your app, see the [AllowFallbackToConsentlessPositions](#) method in the [Geolocator](#) class.

Step 2: Get the user's location and register for changes in location permissions

The [GetGeopositionAsync](#) method performs a one-time reading of the current location. Here, a `switch` statement is used with `accessStatus` (from the previous example) to act only when access to the user's location is allowed. If access to the user's location is

allowed, the code creates a **Geolocator** object, registers for changes in location permissions, and requests the user's location.

C#

```
switch (accessStatus)
{
    case GeolocationAccessStatus.Allowed:
        _rootPage.NotifyUser("Waiting for update...",
            NotifyType.StatusMessage);

        // If DesiredAccuracy or DesiredAccuracyInMeters are not set (or
        // value is 0), DesiredAccuracy.Default is used.
        Geolocator geolocator = new Geolocator { DesiredAccuracyInMeters =
            _desiredAccuracyInMetersValue };

        // Subscribe to the StatusChanged event to get updates of location
        // status changes.
        _geolocator.StatusChanged += OnStatusChanged;

        // Carry out the operation.
        Geoposition pos = await geolocator.GetGeopositionAsync();

        UpdateLocationData(pos);
        _rootPage.NotifyUser("Location updated.", NotifyType.StatusMessage);
        break;

    case GeolocationAccessStatus.Denied:
        _rootPage.NotifyUser("Access to location is denied.",
            NotifyType.ErrorMessage);
        LocationDisabledMessage.Visibility = Visibility.Visible;
        UpdateLocationData(null);
        break;

    case GeolocationAccessStatus.Unspecified:
        _rootPage.NotifyUser("Unspecified error.", NotifyType.ErrorMessage);
        UpdateLocationData(null);
        break;
}
```

Step 3: Handle changes in location permissions

The **Geolocator** object triggers the **StatusChanged** event to indicate that the user's location settings changed. That event passes the corresponding status via the argument's **Status** property (of type **PositionStatus**). Note that this method is not called from the UI thread and the **Dispatcher** object invokes the UI changes.

C#


```

using Windows.UI.Core;
...
async private void OnStatusChanged(Geolocator sender, StatusChangedEventArgs e)
{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        // Show the location setting message only if status is disabled.
        LocationDisabledMessage.Visibility = Visibility.Collapsed;

        switch (e.Status)
        {
            case PositionStatus.Ready:
                // Location platform is providing valid data.
                ScenarioOutput_Status.Text = "Ready";
                _rootPage.NotifyUser("Location platform is ready.",
NotifyType.StatusMessage);
                break;

            case PositionStatus.Initializing:
                // Location platform is attempting to acquire a fix.
                ScenarioOutput_Status.Text = "Initializing";
                _rootPage.NotifyUser("Location platform is attempting to
obtain a position.", NotifyType.StatusMessage);
                break;

            case PositionStatus.NoData:
                // Location platform could not obtain location data.
                ScenarioOutput_Status.Text = "No data";
                _rootPage.NotifyUser("Not able to determine the location.",
NotifyType.ErrorMessage);
                break;

            case PositionStatus.Disabled:
                // The permission to access location data is denied by the
user or other policies.
                ScenarioOutput_Status.Text = "Disabled";
                _rootPage.NotifyUser("Access to location is denied.",
NotifyType.ErrorMessage);

                // Show message to the user to go to location settings.
                LocationDisabledMessage.Visibility = Visibility.Visible;

                // Clear any cached location data.
                UpdateLocationData(null);
                break;

            case PositionStatus.NotInitialized:
                // The location platform is not initialized. This indicates
that the application
                // has not made a request for location data.
                ScenarioOutput_Status.Text = "Not initialized";
                _rootPage.NotifyUser("No request for location is made yet.",
NotifyType.StatusMessage);

```

```

        break;

        case PositionStatus.NotAvailable:
            // The location platform is not available on this version of
the OS.
            ScenarioOutput_Status.Text = "Not available";
            _rootPage.NotifyUser("Location is not available on this
version of the OS.", NotifyType.ErrorMessage);
            break;

        default:
            ScenarioOutput_Status.Text = "Unknown";
            _rootPage.NotifyUser(string.Empty,
NotifyType.StatusMessage);
            break;
    }
});
}

```

Respond to location updates

This section describes how to use the [PositionChanged](#) event to receive updates of the user's location over a period of time. Because the user could revoke access to location at any time, it's important call [RequestAccessAsync](#) and use the [StatusChanged](#) event as shown in the previous section.

This section assumes that you've already enabled the location capability and called [RequestAccessAsync](#) from the UI thread of your foreground app.

Step 1: Define the report interval and register for location updates

In this example, a `switch` statement is used with `accessStatus` (from the previous example) to act only when access to the user's location is allowed. If access to the user's location is allowed, the code creates a [Geolocator](#) object, specifies the tracking type, and registers for location updates.

The [Geolocator](#) object can trigger the [PositionChanged](#) event based on a change in position (distance-based tracking) or a change in time (periodic-based tracking).

- For distance-based tracking, set the [MovementThreshold](#) property.
- For periodic-based tracking, set the [ReportInterval](#) property.

If neither property is set, a position is returned every 1 second (equivalent to `ReportInterval = 1000`). Here, a 2 second (`ReportInterval = 2000`) report interval is

used.

C#

```
using Windows.Devices.Geolocation;
...
var accessStatus = await Geolocator.RequestAccessAsync();

switch (accessStatus)
{
    case GeolocationAccessStatus.Allowed:
        // Create Geolocator and define periodic-based tracking (2 second interval).
        _geolocator = new Geolocator { ReportInterval = 2000 };

        // Subscribe to the PositionChanged event to get location updates.
        _geolocator.PositionChanged += OnPositionChanged;

        // Subscribe to StatusChanged event to get updates of location status changes.
        _geolocator.StatusChanged += OnStatusChanged;

        _rootPage.NotifyUser("Waiting for update...",
            NotifyType.StatusMessage);
        LocationDisabledMessage.Visibility = Visibility.Collapsed;
        StartTrackingButton.IsEnabled = false;
        StopTrackingButton.IsEnabled = true;
        break;

    case GeolocationAccessStatus.Denied:
        _rootPage.NotifyUser("Access to location is denied.",
            NotifyType.ErrorMessage);
        LocationDisabledMessage.Visibility = Visibility.Visible;
        break;

    case GeolocationAccessStatus.Unspecified:
        _rootPage.NotifyUser("Unspecified error!", NotifyType.ErrorMessage);
        LocationDisabledMessage.Visibility = Visibility.Collapsed;
        break;
}
```

Step 2: Handle location updates

The **Geolocator** object triggers the **PositionChanged** event to indicate that the user's location changed or time has passed, depending on how you've configured it. That event passes the corresponding location via the argument's **Position** property (of type **Geoposition**). In this example, the method is not called from the UI thread and the **Dispatcher** object invokes the UI changes.

C#

```

using Windows.UI.Core;
...
async private void OnPositionChanged(Geolocator sender,
PositionChangedEventArgs e)
{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        _rootPage.NotifyUser("Location updated.", NotifyType.StatusMessage);
        UpdateLocationData(e.Position);
    });
}

```

Change the location privacy settings

If the location privacy settings don't allow your app to access the user's location, we recommend that you provide a convenient link to the **location privacy settings** in the **Settings** app. In this example, a Hyperlink control is used to navigate to the `ms-settings:privacy-location` URL.

XAML

```

<!--Set Visibility to Visible when access to location is denied -->
<TextBlock x:Name="LocationDisabledMessage" FontStyle="Italic"
    Visibility="Collapsed" Margin="0,15,0,0" TextWrapping="Wrap">
    <Run Text="This app is not able to access Location. Go to "/>
    <Hyperlink NavigateUri="ms-settings:privacy-location">
        <Run Text="Settings"/>
    </Hyperlink>
    <Run Text=" to check the location privacy settings."/>
</TextBlock>

```

Alternatively, your app can call the [LaunchUriAsync](#) method to launch the **Settings** app from code. For more info, see [Launch the Windows Settings app](#).

C#

```

using Windows.System;
...
bool result = await Launcher.LaunchUriAsync(new Uri("ms-settings:privacy-
location"));

```

Troubleshoot your app

Before your app can access the user's location, **Location** must be enabled on the device. In the **Settings** app, check that the following **location privacy settings** are turned on:

- **Location for this device...** is turned **on** (not applicable in Windows 10 Mobile)
- The location services setting, **Location**, is turned **on**
- Under **Choose apps that can use your location**, your app is set to **on**

Related topics

- [UWP geolocation sample](#) 
- [Design guidelines for geofencing](#)
- [Design guidelines for location-aware apps](#)

Feedback

Was this page helpful?

 **Yes**

 **No**

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Guidelines for location-aware apps

Article • 06/21/2024

This topic describes performance guidelines for apps that require access to a user's location.

Important APIs

- [Geolocation](#)
- [Geolocator](#)

Recommendations

- Start using the location object only when the app requires location data.

Call the [RequestAccessAsync](#) before accessing the user's location. At that time, your app must be in the foreground and [RequestAccessAsync](#) must be called from the UI thread. Until the user grants your app permission to their location, your app can't access location data.
- If location isn't essential to your app, don't access it until the user tries to complete a task that requires it. For example, if a social networking app has a button for "Check in with my location," the app shouldn't access location until the user clicks the button. It's okay to immediately access location if it is required for your app's main function.
- The first use of the [Geolocator](#) object must be made on the main UI thread of the foreground app, to trigger the consent prompt to the user. The first use of the [Geolocator](#) can be either the first call to [getGeopositionAsync](#) or the first registration of a handler for the [positionChanged](#) event.
- Tell the user how location data will be used.
- Provide UI to enable users to manually refresh their location.
- Display a progress bar or ring while waiting to get location data.
- Show appropriate error messages or dialogs when location services are disabled or not available.

If the location settings don't allow your app to access the user's location, we recommend providing a convenient link to the **location privacy settings** in the **Settings** app. For example, you could use a [Hyperlink](#) control or call the

[LaunchUriAsync](#) method to launch the **Settings** app from code using the `ms-settings:privacy-location` URI. For more info, see [Launch the Windows Settings app](#).

- Clear cached location data and release the **Geolocator** when the user disables access to location info.

Release the **Geolocator** object if the user turns off access to location info through Settings. The app will then receive **ACCESS_DENIED** results for any location API calls. If your app saves or caches location data, clear any cached data when the user revokes access to location info. Provide an alternate way to manually enter location info when location data is not available via location services.

- Provide UI for reenabling location services. For example, provide a refresh button that reinstantiates the **Geolocator** object and tries to get location info again.

Have your app provide UI for reenabling location services—

- If the user reenables location access after disabling it, there is no notification to the app. The **status** property does not change and there is no **statusChanged** event. Your app should create a new **Geolocator** object and call [getGeopositionAsync](#) to try to get updated location data, or subscribe again to **positionChanged** events. If the status then indicates that location has been reenabled, clear any UI by which your app previously notified the user that location services were disabled, and respond appropriately to the new status.
- Your app should also try again to get location data upon activation, or when the user explicitly tries to use functionality that requires location info, or at any other scenario-appropriate time.

Performance

- Use one-time location requests if your app doesn't need to receive location updates. For example, an app that adds a location tag to a photo doesn't need to receive location update events. Instead, it should request location using [getGeopositionAsync](#), as described in [Get current location](#).

When you make a one-time location request, you should set the following values.

- Specify the accuracy requested by your app by setting the **DesiredAccuracy** or the **DesiredAccuracyInMeters**. See below for recommendations on using these parameters
- Set the max age parameter of [GetGeopositionAsync](#) to specify how long ago a location can have been obtained to be useful for your app. If your app can use a position that is a few seconds or minutes old, your app can receive a position almost immediately and contribute to saving device power.

- Set the timeout parameter of [GetGeopositionAsync](#). This is how long your app can wait for a position or an error to be returned. You will need to figure out the trade-offs between responsiveness to the user and accuracy your app needs.
- Use continuous location session when frequent position updates are required. Use [positionChanged](#) and [statusChanged](#) events for detecting movement past a specific threshold or for continuous location updates as they occur.

When requesting location updates, you may want to specify the accuracy requested by your app by setting the [DesiredAccuracy](#) or the [DesiredAccuracyInMeters](#). You should also set the frequency at which the location updates are needed, by using the [MovementThreshold](#) or the [ReportInterval](#).

- Specify the movement threshold. Some apps need location updates only when the user has moved a large distance. For example, an app that provides local news or weather updates may not need location updates unless the user's location has changed to a different city. In this case, you adjust the minimum required movement for a location update event by setting the [MovementThreshold](#) property. This has the effect of filtering out [PositionChanged](#) events. These events are raised only when the change in position exceeds the movement threshold.
- Use [reportInterval](#) that aligns with your app experience and that minimizes the use of system resources. For example, a weather app may require a data update only every 15 minutes. Most apps, other than real-time navigation apps, don't require a highly accurate, constant stream of location updates. If your app doesn't require the most accurate stream of data possible, or requires updates infrequently, set the [ReportInterval](#) property to indicate the minimum frequency of location updates that your app needs. The location source can then conserve power by calculating location only when needed.

Apps that do require real-time data should set [ReportInterval](#) to 0, to indicate that no minimum interval is specified. The default report interval is 1 second or as frequent as the hardware can support – whichever is shorter.

Devices that provide location data may track the report interval requested by different apps, and provide data reports at the smallest requested interval. The app with the greatest need for accuracy thus receives the data it needs.

Therefore, it's possible that the location provider will generate updates at a higher frequency than your app requested, if another app has requested more frequent updates.

Note It isn't guaranteed that the location source will honor the request for the given report interval. Not all location provider devices track the report interval, but you should still provide it for those that do.

- To help conserve power, set the **desiredAccuracy** property to indicate to the location platform whether your app needs high-accuracy data. If no apps require high-accuracy data, the system can save power by not turning on GPS providers.
 - Set **desiredAccuracy** to **HIGH** to enable the GPS to acquire data.
 - Set **desiredAccuracy** to **Default** and use only a single-shot call pattern to minimize power consumption if your app uses location info solely for ad targeting.

If your app has specific needs around accuracy, you may want to use the **DesiredAccuracyInMeters** property instead of using **DesiredAccuracy**. This is particularly useful on Windows Phone, where position can usually be obtained based on cellular beacons, Wi-Fi beacons and satellites. Picking a more specific accuracy value will help the system identify the right technologies to use with the lowest power cost when providing a position.

For example:

- If your app is obtaining location for ads tuning, weather, news, etc, an accuracy of 5000 meter is generally enough.
- If your app is displaying nearby deals in the neighborhood, an accuracy of 300 meter is generally good to provide results.
- If the user is looking for recommendations to nearby restaurants, we likely want to get a position within a block, so an accuracy of 100 meters is sufficient.
- If the user is trying to share his position, the app should request an accuracy of about 10 meters.
- Use the **Geocoordinate.accuracy** property if your app has specific accuracy requirements. For example, navigation apps should use the **Geocoordinate.accuracy** property to determine whether the available location data meets the app's requirements.
- Consider start-up delay. The first time an app requests location data, there might be a short delay (1-2 seconds) while the location provider starts up. Consider this in the design of your app's UI. For instance, you may want to avoid blocking other tasks pending the completion of the call to **GetGeopositionAsync**.
- Consider background behavior. If your app doesn't have focus, it won't receive location update events while it's suspended in the background. If your app tracks

location updates by logging them, be aware of this. When the app regains focus, it receives only new events. It does not get any updates that occurred when it was inactive.

- Use raw and fusion sensors efficiently. There are two types of sensors: *raw* and *fusion*.
 - Raw sensors include the accelerometer, gyrometer, and magnetometer.
 - Fusion sensors include orientation, inclinometer, and compass. Fusion sensors get their data from combinations of the raw sensors.

The Windows Runtime APIs can access all of these sensors except for the magnetometer. Fusion sensors are more accurate and stable than raw sensors, but they use more power. You should use the right sensors for the right purpose. For more info, see [Sensors](#).

Connected standby

- When the PC is in connected standby state, [Geolocator](#) objects can always be instantiated. However, the [Geolocator](#) object will not find any sensors to aggregate and therefore calls to [GetGeopositionAsync](#) will time out after 7 seconds, [PositionChanged](#) event listeners will never be called, and [StatusChanged](#) event listeners will be called once with the **NoData** status.

Additional usage guidance

Detecting changes in location settings

The user can turn off location functionality by using the **location privacy settings** in the **Settings** app.

- To detect when the user disables or reenables location services:
 - Handle the [StatusChanged](#) event. The [Status](#) property of the argument to the [StatusChanged](#) event has the value **Disabled** if the user turns off location services.
 - Check the error codes returned from [GetGeopositionAsync](#). If the user has disabled location services, calls to [GetGeopositionAsync](#) fail with an **ACCESS_DENIED** error and the [LocationStatus](#) property has the value **Disabled**.
- If you have an app for which location data is essential—for example, a mapping app—, be sure to do the following:
 - Handle the [PositionChanged](#) event to get updates if the user's location changes.

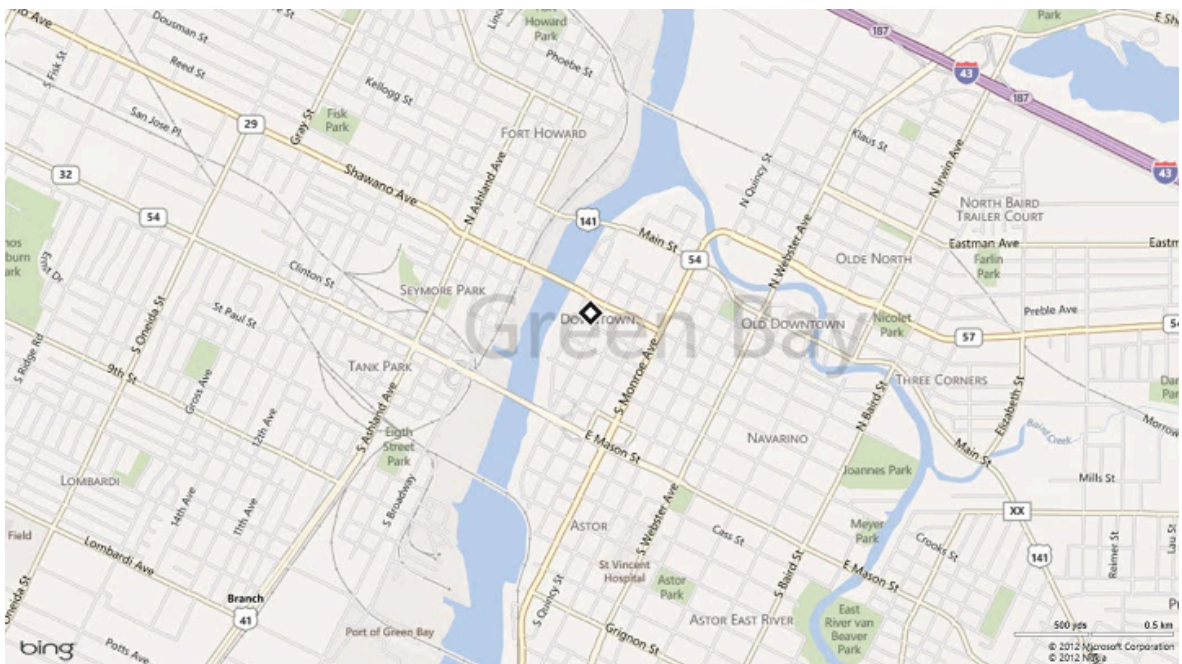
- Handle the [StatusChanged](#) event as described previously, to detect changes in location settings.

Note that the location service will return data as it becomes available. It may first return a location with a larger error radius and then update the location with more accurate information as it becomes available. Apps displaying the user's location would normally want to update the location as more accurate information becomes available.

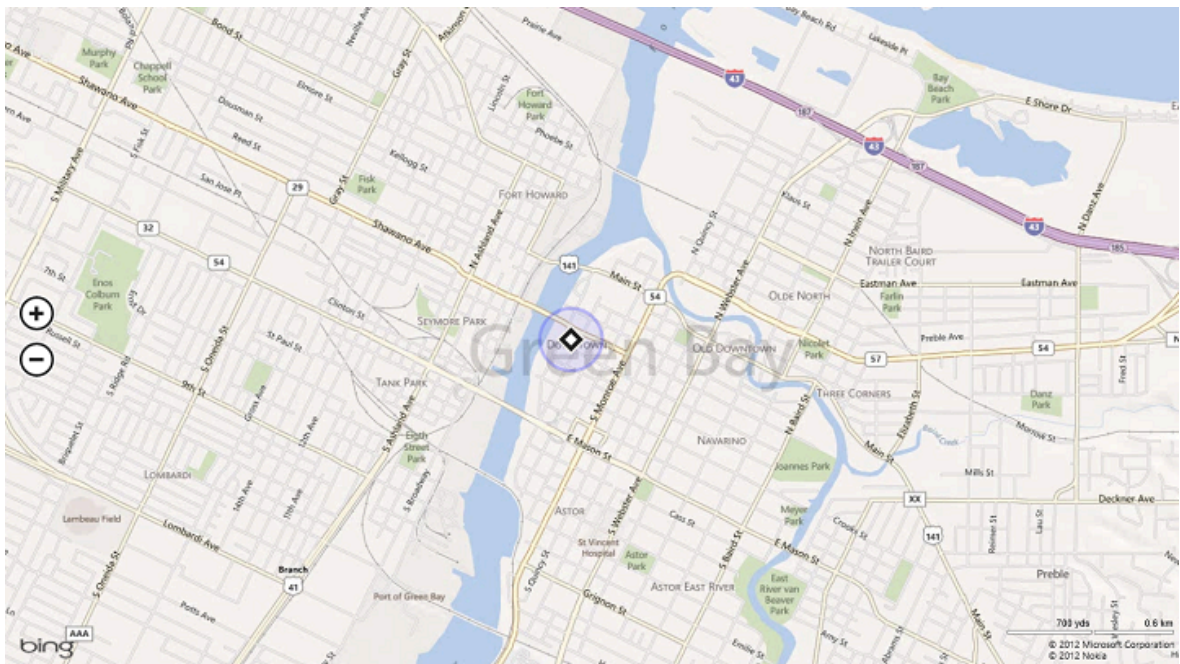
Graphical representations of location

Have your app use [Geocoordinate.accuracy](#) to denote the user's current location on the map clearly. There are three main bands for accuracy—an error radius of approximately 10 meters, an error radius of approximately 100 meters, and an error radius of greater than 1 kilometer. By using the accuracy information, you can ensure that your app displays location accurately in the context of the data available. For general information about using the map control, see [Display maps with 2D, 3D, and Streetside views](#).

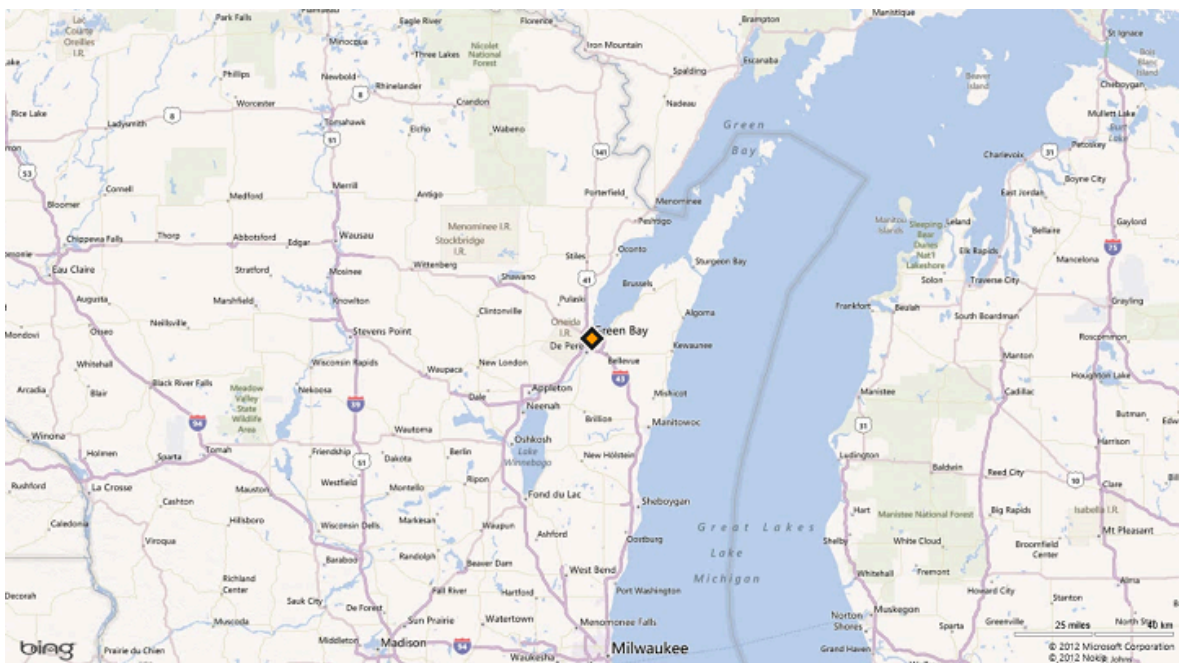
- For accuracy approximately equal to 10 meters (GPS resolution), location can be denoted by a dot or pin on the map. With this accuracy, latitude-longitude coordinates and street address can be shown as well.



- For accuracy between 10 and 500 meters (approximately 100 meters), location is generally received through Wi-Fi resolution. Location obtained from cellular has an accuracy of around 300 meters. In such a case, we recommend that your app show an error radius. For apps that show directions where a centering dot is required, such a dot can be shown with an error radius surrounding it.



- If the accuracy returned is greater than 1 kilometer, you are probably receiving location info at IP-level resolution. This level of accuracy is often too low to pinpoint a particular spot on a map. Your app should zoom in to the city level on the map, or to the appropriate area based on the error radius (for example, region level).



When location accuracy switches from one band of accuracy to another, provide a graceful transition between the different graphical representations. This can be done by:

- Making the transition animation smooth and keeping the transition fast and fluid.
- Waiting for a few consecutive reports to confirm the change in accuracy, to help prevent unwanted and too-frequent zooms.

Textual representations of location

Some types of apps—for example, a weather app or a local information app—need ways to represent location textually at the different bands of accuracy. Be sure to display the location clearly and only down to the level of accuracy provided in the data.

- For accuracy approximately equal to 10 meters (GPS resolution), the location data received is fairly accurate and so can be communicated to the level of the neighborhood name. City name, state or province name, and country/region name can also be used.
- For accuracy approximately equal to 100 meters (Wi-Fi resolution), the location data received is moderately accurate and so we recommend that you display information down to the city name. Avoid using the neighborhood name.
- For accuracy greater than 1 kilometer (IP resolution), display only the state or province, or country/region name.

Privacy considerations

A user's geographic location is personally identifiable information (PII). The following website provides guidance for protecting user privacy.

- [Microsoft Privacy](#) 

Related topics

- [Set up a geofence](#)
- [Get current location](#)
- [Display maps with 2D, 3D, and Streetside views](#)
- [UWP location sample \(geolocation\)](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Guidelines for using Visits tracking

Article • 06/21/2024

The Visits feature streamlines the process of location tracking to make it more efficient for the practical purposes of many apps. A Visit is defined as a significant geographical area that the user enters and exits. Visits are similar to [geofences](#) in that they allow the app to be notified only when the user enters or exits certain areas of interest, eliminating the need for continual location tracking which can be a drain on battery life. However, unlike geofences, Visit areas are dynamically identified at the platform level and do not need to be defined explicitly by individual apps. Also, the selection of which Visits an app will track is handled by a single granularity setting, rather than by subscribing to individual places.

Preliminary setup

Before going further, make sure your app is capable of accessing the device's location. You will need to declare the `Location` capability in the manifest and call the [Geolocator.RequestAccessAsync](#) method to ensure that users give the app location permissions. See [Get the user's location](#) for more information on how to do this.

Remember to add the `Geolocation` namespace to your class. This will be needed for all of the code snippets in this guide to work.

C#

```
using Windows.Devices.Geolocation;
```

Check the latest Visit

The simplest way to use the Visits tracking feature is to retrieve the last known Visit-related state change. A state change is a platform-logged event in which either the user enters/exits a location of significance, there is significant movement since the last report, or the user's location is lost (see the [VisitStateChange](#) enum). State changes are represented by [Geovisit](#) instances. To retrieve the [Geovisit](#) instance for the last recorded state change, simply use the designated method in the [GeovisitMonitor](#) class.

📌 Note

Checking the last logged Visit does not guarantee that Visits are currently being tracked by the system. In order to track Visits as they happen, you must either be monitoring them in the foreground or register for background tracking (see sections below).

C#

```
private async void GetLatestStateChange() {  
    // retrieve the Geovisit instance  
    Geovisit latestVisit = await GeovisitMonitor.GetLastReportAsync();  
  
    // Using the properties of "latestVisit", parse out the time that the  
    state  
    // change was recorded, the device's location when the change was  
    recorded,  
    // and the type of state change.  
}
```

Parse a Geovisit instance (optional)

The following method converts all of the information stored in a **Geovisit** instance into an easily readable string. It can be used in any of the scenarios in this guide to help provide feedback for the Visits being reported.

C#

```
private string ParseGeovisit(Geovisit visit){  
    string visitString = null;  
  
    // Use a DateTimeFormatter object to process the timestamp. The  
    following  
    // format configuration will give an intuitive representation of the  
    date/time  
    Windows.Globalization.DateTimeFormatting.DateTimeFormatter  
    formatterLongTime;  
  
    formatterLongTime = new  
    Windows.Globalization.DateTimeFormatting.DateTimeFormatter(  
        "{hour.integer}:{minute.integer(2)}:{second.integer(2)}", new[] {  
        "en-US" }, "US",  
        Windows.Globalization.CalendarIdentifiers.Gregorian,  
        Windows.Globalization.ClockIdentifiers.TwentyFourHour);  
  
    // use this formatter to convert the timestamp to a string, and add to  
    "visitString"  
    visitString = formatterLongTime.Format(visit.Timestamp);  
  
    // Next, add on the state change type value  
    visitString += " " + visit.StateChange.ToString();  
}
```

```

        // Next, add the position information (if any is provided; this will be
        null if
        // the reported event was "TrackingLost")
        if (visit.Position != null) {
            visitString += " (" +
            visit.Position.Coordinate.Point.Position.Latitude.ToString() + "," +
            visit.Position.Coordinate.Point.Position.Longitude.ToString() +
            ")";
        }

        return visitString;
    }
}

```

Monitor Visits in the foreground

The **GeovisitMonitor** class used in the previous section also handles the scenario of listening for state changes over a period of time. You can do this by instantiating this class, registering a handler method for its event, and calling the **Start** method.

C#

```

// this GeovisitMonitor instance will belong to the class scope
GeovisitMonitor monitor;

public void RegisterForVisits() {

    // Create and initialize a new monitor instance.
    monitor = new GeovisitMonitor();

    // Attach a handler to receive state change notifications.
    monitor.VisitStateChanged += OnVisitStateChanged;

    // Calling the start method will start Visits tracking for a specified
    scope:
    // For higher granularity such as venue/building level changes, choose
    Venue.
    // For lower granularity in the range of zipcode level changes, choose
    City.
    monitor.Start(VisitMonitoringScope.Venue);
}

```

In this example, the **OnVisitStateChanged** method will handle incoming Visit reports. The corresponding **Geovisit** instance is passed in through the event parameter.

C#

```

private void OnVisitStateChanged(GeoVisitWatcher sender,
    GeoVisitStateChangedEventArgs args) {

```



```
Geovisit visit = args.Visit;

// Using the properties of "visit", parse out the time that the state
// change was recorded, the device's location when the change was
// recorded,
// and the type of state change.
}
```

When the app is finished monitoring for Visit-related state changes, it should stop the monitor and unregister the event handler(s). This should also be done whenever the app is suspended or closed.

```
C#

public void UnregisterFromVisits() {

    // Stop the monitor to stop tracking Visits. Otherwise, tracking will
    // continue until the monitor instance is destroyed.
    monitor.Stop();

    // Remove the handler to stop receiving state change events.
    monitor.VisitStateChanged -= OnVisitStateChanged;
}
```

Monitor Visits in the background

You can also implement Visit monitoring in a background task, so that Visit-related activity can be handled on the device even when your app isn't open. This is the recommended method, as it is more versatile and energy-efficient.

This guide will use the model in [Create and register an out-of-process background task](#), in which the main application files live in one project and the background task file lives in a separate project in the same solution. If you are new to implementing background tasks, it is recommended that you follow that guidance primarily, making the necessary substitutions below to create a Visit-handling background task.

ⓘ Note

In the following snippets, some important functionality such as error handling and local storage is absent for the sake of simplicity. For a robust implementation of background Visits handling, see the [sample app](#).

First, make sure your app has declared background task permissions. In the `Application/Extensions` element of your *Package.appxmanifest* file, add the following

extension (add an `Extensions` element if one does not already exist).

XML

```
<Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.VisitBackgroundTask">
  <BackgroundTasks>
    <Task Type="location" />
  </BackgroundTasks>
</Extension>
```

Next, in the definition of the background task class, paste in the following code. The `Run` method of this background task will simply pass the trigger details (which contain the Visits information) into a separate method.

C#

```
using Windows.ApplicationModel.Background;

namespace Tasks {

    public sealed class VisitBackgroundTask : IBackgroundTask {

        public void Run(IBackgroundTaskInstance taskInstance) {

            // get a deferral
            BackgroundTaskDeferral deferral = taskInstance.GetDeferral();

            // this task's trigger will be a Geovisit trigger
            GeovisitTriggerDetails triggerDetails =
            taskInstance.TriggerDetails as GeovisitTriggerDetails;

            // Handle Visit reports
            GetVisitReports(triggerDetails);

            finally {
                deferral.Complete();
            }
        }
    }
}
```

Define the `GetVisitReports` method somewhere in this same class.

C#

```
private void GetVisitReports(GeovisitTriggerDetails triggerDetails) {

    // Read reports from the triggerDetails. This populates the "reports"
    variable
}
```

```

        // with all of the Geovisit instances that have been logged since the
        previous
        // report reading.
        IReadOnlyList<Geovisit> reports = triggerDetails.ReadReports();

        foreach (Geovisit report in reports) {
            // Using the properties of "visit", parse out the time that the
            state
            // change was recorded, the device's location when the change was
            recorded,
            // and the type of state change.
        }

        // Note: depending on the intent of the app, you may wish to store the
        // reports in the app's local storage so they can be retrieved the next
        time
        // the app is opened in the foreground.
    }

```

Next, in the main project of your app, you'll need to carry out the registration of this background task. Create a registering method that can be called by some user action or is called each time the class is activated.

C#

```

// a reference to this registration should be declared at the class level
private IBackgroundTaskRegistration visitTask = null;

// The app must call this method at some point to allow future use of
// the background task.
private async void RegisterBackgroundTask(object sender, RoutedEventArgs e)
{
    string taskName = "MyVisitTask";
    string taskEntryPoint = "Tasks.VisitBackgroundTask";

    // First check whether the task in question is already registered
    foreach (var task in BackgroundTaskRegistration.AllTasks) {
        if (task.Value.Name == taskName) {
            // if a task is found with the name of this app's background
            task, then
            // return and do not attempt to register this task
            return;
        }
    }

    // Attempt to register the background task.
    try {
        // Get permission for a background task from the user. If the user
        has
        // already responded once, this does nothing and the user must
        manually
        // update their preference via Settings.
    }
}

```

```

        BackgroundAccessStatus backgroundAccessStatus = await
BackgroundExecutionManager.RequestAccessAsync();

        switch (backgroundAccessStatus) {
            case BackgroundAccessStatus.AlwaysAllowed:
            case BackgroundAccessStatus.AllowedSubjectToSystemPolicy:
                // BackgroundTask is allowed
                break;

            default:
                // notify user that background tasks are disabled for this
app
                //...
                break;
        }

        // Create a new background task builder
        BackgroundTaskBuilder visitTaskBuilder = new
BackgroundTaskBuilder();

        visitTaskBuilder.Name = exampleTaskName;
        visitTaskBuilder.TaskEntryPoint = taskEntryPoint;

        // Create a new Visit trigger
        var trigger = new GeovisitTrigger();

        // Set the desired monitoring scope.
        // For higher granularity such as venue/building level changes,
choose Venue.
        // For lower granularity in the range of zipcode level changes,
choose City.
        trigger.MonitoringScope = VisitMonitoringScope.Venue;

        // Associate the trigger with the background task builder
        visitTaskBuilder.SetTrigger(trigger);

        // Register the background task
        visitTask = visitTaskBuilder.Register();
    }
    catch (Exception ex) {
        // notify user that the task failed to register, using ex.ToString()
    }
}

```

This establishes that a background task class called `VisitBackgroundTask` in the namespace `Tasks` will do something with the `location` trigger type.

Your app should now be capable of registering the Visits-handling background task, and this task should be activated whenever the device logs a Visit-related state change. You will need to fill in the logic in your background task class to determine what to do with this state change information.

Related topics

- [Create and register an out-of-process background task](#)
- [Get the user's location](#)
- [Windows.Devices.Geolocation namespace](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Set up a geofence

Article • 06/21/2024

Set up a [Geofence](#) in your app, and learn how to handle notifications in the foreground and background.

Enable the location capability

1. In **Solution Explorer**, double-click on **package.appxmanifest** and select the **Capabilities** tab.
2. In the **Capabilities** list, check **Location**. This adds the **Location** device capability to the package manifest file.

XML

```
<Capabilities>
  <!-- DeviceCapability elements must follow Capability elements (if
present) -->
  <DeviceCapability Name="location"/>
</Capabilities>
```

Geofence set up

Step 1: Request access to the user's location

Important

You must request access to the user's location by using the [RequestAccessAsync](#) method before attempting to access the user's location. You must call the **RequestAccessAsync** method from the UI thread and your app must be in the foreground. Your app will not be able to access the user's location information until after the user grants permission to your app.

C#

```
using Windows.Devices.Geolocation;
...
var accessStatus = await Geolocator.RequestAccessAsync();
```

The [RequestAccessAsync](#) method prompts the user for permission to access their location. The user is only prompted once (per app). After the first time they grant or deny permission, this method no longer prompts the user for permission. To help the user change location permissions after they've been prompted, we recommend that you provide a link to the location settings as demonstrated later in this topic.

Step 2: Register for changes in geofence state and location permissions

In this example, a **switch** statement is used with **accessStatus** (from the previous example) to act only when access to the user's location is allowed. If access to the user's location is allowed, the code accesses the current geofences, registers for geofence state changes, and registers for changes in location permissions.

Tip When using a geofence, monitor changes in location permissions using the [StatusChanged](#) event from the [GeofenceMonitor](#) class instead of the [StatusChanged](#) event from the [Geolocator](#) class. A [GeofenceMonitorStatus](#) of **Disabled** is equivalent to a disabled [PositionStatus](#) - both indicate that the app does not have permission to access the user's location.

C#

```
switch (accessStatus)
{
    case GeolocationAccessStatus.Allowed:
        geofences = GeofenceMonitor.Current.Geofences;

        FillRegisteredGeofenceListBoxWithExistingGeofences();
        FillEventListBoxWithExistingEvents();

        // Register for state change events.
        GeofenceMonitor.Current.GeofenceStateChanged +=
OnGeofenceStateChanged;
        GeofenceMonitor.Current.StatusChanged += OnGeofenceStatusChanged;
        break;

    case GeolocationAccessStatus.Denied:
        _rootPage.NotifyUser("Access denied.", NotifyType.ErrorMessage);
        break;

    case GeolocationAccessStatus.Unspecified:
        _rootPage.NotifyUser("Unspecified error.", NotifyType.ErrorMessage);
        break;
}
```

Then, when navigating away from your foreground app, unregister the event listeners.

C#

```
protected override void OnNavigatingFrom(NavigatingCancelEventArgs e)
{
    GeofenceMonitor.Current.GeofenceStateChanged -= OnGeofenceStateChanged;
    GeofenceMonitor.Current.StatusChanged -= OnGeofenceStatusChanged;

    base.OnNavigatingFrom(e);
}
```

Step 3: Create the geofence

Now, you are ready to define and set up a **Geofence** object. There are several different constructor overloads to choose from, depending on your needs. In the most basic geofence constructor, specify only the **Id** and the **Geoshape** as shown here.

C#

```
// Set the fence ID.
string fenceId = "fence1";

// Define the fence location and radius.
BasicGeoposition position;
position.Latitude = 47.6510;
position.Longitude = -122.3473;
position.Altitude = 0.0;
double radius = 10; // in meters

// Set a circular region for the geofence.
Geocircle geocircle = new Geocircle(position, radius);

// Create the geofence.
Geofence geofence = new Geofence(fenceId, geocircle);
```

You can fine-tune your geofence further by using one of the other constructors. In the next example, the geofence constructor specifies these additional parameters:

- **MonitoredStates** - Indicates what geofence events you want to receive notifications for entering the defined region, leaving the defined region, or removal of the geofence.
- **SingleUse** - Removes the geofence once all the states the geofence is being monitored for have been met.
- **DwellTime** - Indicates how long the user must be in or out of the defined area before the enter/exit events are triggered.
- **StartTime** - Indicates when to start monitoring the geofence.

- **Duration** - Indicates the period for which to monitor the geofence.

C#

```
// Set the fence ID.
string fenceId = "fence2";

// Define the fence location and radius.
BasicGeoposition position;
position.Latitude = 47.6510;
position.Longitude = -122.3473;
position.Altitude = 0.0;
double radius = 10; // in meters

// Set the circular region for geofence.
Geocircle geocircle = new Geocircle(position, radius);

// Remove the geofence after the first trigger.
bool singleUse = true;

// Set the monitored states.
MonitoredGeofenceStates monitoredStates =
    MonitoredGeofenceStates.Entered |
    MonitoredGeofenceStates.Exited |
    MonitoredGeofenceStates.Removed;

// Set how long you need to be in geofence for the enter event to fire.
TimeSpan dwellTime = TimeSpan.FromMinutes(5);

// Set how long the geofence should be active.
TimeSpan duration = TimeSpan.FromDays(1);

// Set up the start time of the geofence.
DateTimeOffset startTime = DateTime.Now;

// Create the geofence.
Geofence geofence = new Geofence(fenceId, geocircle, monitoredStates,
    singleUse, dwellTime, startTime, duration);
```

After creating, remember to register your new **Geofence** to the monitor.

C#

```
// Register the geofence
try {
    GeofenceMonitor.Current.Geofences.Add(geofence);
} catch {
    // Handle failure to add geofence
}
```

Step 4: Handle changes in location permissions

The **GeofenceMonitor** object triggers the **StatusChanged** event to indicate that the user's location settings changed. That event passes the corresponding status via the argument's **sender.Status** property (of type **GeofenceMonitorStatus**). Note that this method is not called from the UI thread and the **Dispatcher** object invokes the UI changes.

C#

```
using Windows.UI.Core;
...
public async void OnGeofenceStatusChanged(GeofenceMonitor sender, object e)
{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        // Show the location setting message only if the status is disabled.
        LocationDisabledMessage.Visibility = Visibility.Collapsed;

        switch (sender.Status)
        {
            case GeofenceMonitorStatus.Ready:
                _rootPage.NotifyUser("The monitor is ready and active.",
                NotifyType.StatusMessage);
                break;

            case GeofenceMonitorStatus.Initializing:
                _rootPage.NotifyUser("The monitor is in the process of initializing.",
                NotifyType.StatusMessage);
                break;

            case GeofenceMonitorStatus.NoData:
                _rootPage.NotifyUser("There is no data on the status of the monitor.",
                NotifyType.ErrorMessage);
                break;

            case GeofenceMonitorStatus.Disabled:
                _rootPage.NotifyUser("Access to location is denied.",
                NotifyType.ErrorMessage);

                // Show the message to the user to go to the location settings.
                LocationDisabledMessage.Visibility = Visibility.Visible;
                break;

            case GeofenceMonitorStatus.NotInitialized:
                _rootPage.NotifyUser("The geofence monitor has not been initialized.",
                NotifyType.StatusMessage);
                break;

            case GeofenceMonitorStatus.NotAvailable:
                _rootPage.NotifyUser("The geofence monitor is not available.",
                NotifyType.ErrorMessage);
```

```

        break;

    default:
        ScenarioOutput_Status.Text = "Unknown";
        _rootPage.NotifyUser(string.Empty, NotifyType.StatusMessage);
        break;
    }
    });
}

```

Set up foreground notifications

After your geofences are created, you must add the logic to handle what happens when a geofence event occurs. Depending on the [MonitoredStates](#) that you have set up, you may receive an event when:

- The user enters a region of interest.
- The user leaves a region of interest.
- The geofence has expired or been removed. Note that a background app is not activated for a removal event.

You can listen for events directly from your app when it is running or register for a background task so that you receive a background notification when an event occurs.

Step 1: Register for geofence state change events

For your app to receive a foreground notification of a geofence state change, you must register an event handler. This is typically set up when you create the geofence.

```

C#

private void Initialize()
{
    // Other initialization logic

    GeofenceMonitor.Current.GeofenceStateChanged += OnGeofenceStateChanged;
}

```

Step 2: Implement the geofence event handler

The next step is to implement the event handlers. The action taken here depends on what your app is using the geofence for.

C#

```
public async void OnGeofenceStateChanged(GeofenceMonitor sender, object e)
{
    var reports = sender.ReadReports();

    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        foreach (GeofenceStateChangeReport report in reports)
        {
            GeofenceState state = report.NewState;

            Geofence geofence = report.Geofence;

            if (state == GeofenceState.Removed)
            {
                // Remove the geofence from the geofences collection.
                GeofenceMonitor.Current.Geofences.Remove(geofence);
            }
            else if (state == GeofenceState.Entered)
            {
                // Your app takes action based on the entered event.

                // NOTE: You might want to write your app to take a
particular
                // action based on whether the app has internet
connectivity.

            }
            else if (state == GeofenceState.Exited)
            {
                // Your app takes action based on the exited event.

                // NOTE: You might want to write your app to take a
particular
                // action based on whether the app has internet
connectivity.

            }
        }
    });
}
```

Set up background notifications

After your geofences are created, you must add the logic to handle what happens when a geofence event occurs. Depending on the [MonitoredStates](#) that you have set up, you may receive an event when:

- The user enters a region of interest.

- The user leaves a region of interest.
- The geofence has expired or been removed. Note that a background app is not activated for a removal event.

To listen for a geofence event in the background

- Declare the background task in your app's manifest.
- Register the background task in your app. If your app needs internet access, say for accessing a cloud service, you can set a flag for that when the event is triggered. You can also set a flag to make sure that the user is present when the event is triggered so that you are sure that the user gets notified.
- While your app is running in the foreground, prompt the user to grant your app location permissions.

Step 1: Register for geofence state changes

In your app's manifest, under the **Declarations** tab, add a declaration for a location background task. To do this:

- Add a declaration of type **Background Tasks**.
- Set a property task type of **Location**.
- Set an entry point into your app to call when the event is triggered.

Step 2: Register the background task

The code in this step registers the geofencing background task. Recall that when the geofence was created, we checked for location permissions.

C#

```
async private void RegisterBackgroundTask(object sender, RoutedEventArgs e)
{
    // Get permission for a background task from the user. If the user has
    // already answered once,
    // this does nothing and the user must manually update their preference
    // via PC Settings.
    BackgroundAccessStatus backgroundAccessStatus = await
    BackgroundExecutionManager.RequestAccessAsync();

    // Regardless of the answer, register the background task. Note that the
    // user can use
    // the Settings app to prevent your app from running background tasks.
    // Create a new background task builder.
    BackgroundTaskBuilder geofenceTaskBuilder = new BackgroundTaskBuilder();

    geofenceTaskBuilder.Name = SampleBackgroundTaskName;
```

```

geofenceTaskBuilder.TaskEntryPoint = SampleBackgroundTaskEntryPoint;

// Create a new location trigger.
var trigger = new LocationTrigger(LocationTriggerType.Geofence);

// Associate the location trigger with the background task builder.
geofenceTaskBuilder.SetTrigger(trigger);

// If it is important that there is user presence and/or
// internet connection when OnCompleted is called
// the following could be called before calling Register().
// SystemCondition condition = new
SystemCondition(SystemConditionType.UserPresent |
SystemConditionType.InternetAvailable);
// geofenceTaskBuilder.AddCondition(condition);

// Register the background task.
geofenceTask = geofenceTaskBuilder.Register();

// Associate an event handler with the new background task.
geofenceTask.Completed += new
BackgroundTaskCompletedEventHandler(OnCompleted);

BackgroundTaskState.RegisterBackgroundTask(BackgroundTaskState.LocationTrigg
erBackgroundTaskName);

switch (backgroundAccessStatus)
{
case BackgroundAccessStatus.Unspecified:
case BackgroundAccessStatus.Denied:
    rootPage.NotifyUser("This app is not allowed to run in the
background.", NotifyType.ErrorMessage);
    break;
}
}

```

Step 3: Handling the background notification

The action that you take to notify the user depends on what your app does, but you can display a toast notification, play an audio sound, or update a live tile. The code in this step handles the notification.

C#

```

async private void OnCompleted(IBackgroundTaskRegistration sender,
BackgroundTaskCompletedEventArgs e)
{

```

```

if (sender != null)
{
    // Update the UI with progress reported by the background task.
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        try
        {
            // If the background task threw an exception, display the
exception in
            // the error text box.
            e.CheckResult();

            // Update the UI with the completion status of the
background task.
            // The Run method of the background task sets the
LocalSettings.
            var settings = ApplicationData.Current.LocalSettings;

            // Get the status.
            if (settings.Values.ContainsKey("Status"))
            {
                rootPage.NotifyUser(settings.Values["Status"].ToString(),
NotifyType.StatusMessage);
            }

            // Do your app work here.

        }
        catch (Exception ex)
        {
            // The background task had an error.
            rootPage.NotifyUser(ex.ToString(), NotifyType.ErrorMessage);
        }
    });
}
}

```

Change the privacy settings

If the location privacy settings don't allow your app to access the user's location, we recommend that you provide a convenient link to the **location privacy settings** in the **Settings** app. In this example, a Hyperlink control is used navigate to the `ms-settings:privacy-location` URL.

XAML

```

<!--Set Visibility to Visible when access to the user's location is denied.
-->
<TextBlock x:Name="LocationDisabledMessage" FontStyle="Italic"
            Visibility="Collapsed" Margin="0,15,0,0"
TextWrapping="Wrap" >
    <Run Text="This app is not able to access Location. Go to " />
    <Hyperlink NavigateUri="ms-settings:privacy-location">
        <Run Text="Settings" />
    </Hyperlink>
    <Run Text=" to check the location privacy settings."/>
</TextBlock>

```

Alternatively, your app can call the [LaunchUriAsync](#) method to launch the **Settings** app from code. For more info, see [Launch the Windows Settings app](#).

C#

```

using Windows.System;
...
bool result = await Launcher.LaunchUriAsync(new Uri("ms-settings:privacy-
location"));

```

Test and debug your app

Testing and debugging geofencing apps can be a challenge because they depend on a device's location. Here, we outline several methods for testing both foreground and background geofences.

To debug a geofencing app

1. Physically move the device to new locations.
2. Test entering a geofence by creating a geofence region that includes your current physical location, so you're already inside the geofence and the "geofence entered" event is triggered immediately.
3. Use the Microsoft Visual Studio emulator to simulate locations for the device.

Test and debug a geofencing app that is running in the foreground

To test your geofencing app that is running in the foreground

1. Build your app in Visual Studio.
2. Launch your app in the Visual Studio emulator.

3. Use these tools to simulate various locations inside and outside of your geofence region. Be sure to wait long enough past the time specified by the [DwellTime](#) property to trigger the event. Note that you must accept the prompt to enable location permissions for the app. For more info about simulating locations, see [Set the simulated geolocation of the device](#).
4. You can also use the emulator to estimate the size of fences and dwell times approximately needed to be detected at different speeds.

Test and debug a geofencing app that is running in the background

To test your geofencing app that is running the background

1. Build your app in Visual Studio. Note that your app should set the **Location** background task type.
2. Deploy the app locally first.
3. Close your app that is running locally.
4. Launch your app in the Visual Studio emulator. Note that background geofencing simulation is supported on only one app at a time within the emulator. Do not launch multiple geofencing apps within the emulator.
5. From the emulator, simulate various locations inside and outside of your geofence region. Be sure to wait long enough past the [DwellTime](#) to trigger the event. Note that you must accept the prompt to enable location permissions for the app.
6. Use Visual Studio to trigger the location background task. For more info about triggering background tasks in Visual Studio, see [How to trigger background tasks](#).

Troubleshoot your app

Before your app can access location, **Location** must be enabled on the device. In the **Settings** app, check that the following **location privacy settings** are turned on:

- **Location for this device...** is turned **on** (not applicable in Windows 10 Mobile)
- The location services setting, **Location**, is turned **on**
- Under **Choose apps that can use your location**, your app is set to **on**

Related topics

- [UWP geolocation sample](#) 
- [Design guidelines for geofencing](#)
- [Design guidelines for location-aware apps](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Guidelines for geofencing apps

Article • 06/21/2024

Follow these best practices for [geofencing](#) in your app.

Important APIs

- [Geofence class \(XAML\)](#)
- [Geolocator class \(XAML\)](#)

Recommendations

- If your app will need internet access when a [Geofence](#) event occurs, check for internet access before creating the geofence.
 - If the app doesn't currently have internet access, you can prompt the user to connect to the internet before you set up the geofence.
 - If internet access isn't possible, avoid consuming the power required for the geofencing location checks.
- Ensure the relevance of geofencing notifications by checking the time stamp and current location when a geofence event indicates changes to an [Entered](#) or [Exited](#) state. See [Checking the time stamp and current location](#) below for more information.
- Create exceptions to manage cases when a device can't access location info, and notify the user if necessary. Location info may be unavailable because permissions are turned off, the device doesn't contain a GPS radio, the GPS signal is blocked, or the Wi-Fi signal isn't strong enough.
- In general, it isn't necessary to listen for geofence events in the foreground and background at the same time. However, if your app needs to listen for geofence events in both the foreground and background:
 - Call the [ReadReports](#) method to find out if an event has occurred.
 - Unregister your foreground event listener when your app isn't visible to the user and re-register when it becomes visible again.

See [Background and foreground listeners](#) for code examples and more information.

- Don't use more than 1000 geofences per app. The system actually supports thousands of geofences per app, you can maintain good app performance to help reduce the app's memory usage by using no more than 1000.

- Don't create a geofence with a radius smaller than 50 meters. If your app needs to use a geofence with a small radius, advise users to use your app on a device with a GPS radio to ensure the best performance.

Additional usage guidance

Checking the time stamp and current location

When an event indicates a change to an [Entered](#) or [Exited](#) state, check both the time stamp of the event and your current location. Various factors, such as the system not having enough resources to launch a background task, the user not noticing the notification, or the device being in standby, may affect when the event is actually processed by the user. For example, the following sequence may occur:

- Your app creates a geofence and monitors the geofence for enter and exit events.
- The user moves the device inside of the geofence, causing an enter event to be triggered.
- Your app sends a notification to the user that they are now inside the geofence.
- The user was busy and does not notice the notification until 10 minutes later.
- During that 10 minute delay, the user has moved back outside of the geofence.

From the timestamp, you can tell that the action occurred in the past. From the current location, you can see that the user is now back outside of the geofence. Depending on the functionality of your app, you may want to filter out this event.

Background and foreground listeners

In general, your app doesn't need to listen for [Geofence](#) events both in the foreground and in a background task at the same time. The cleanest method for handling a case where you might need both is to let the background task handle the notifications. If you do set up both foreground and background geofence listeners, there is no guarantee which will be triggered first and so you must always call the [ReadReports](#) method to find out if an event has occurred.

If you have set up both foreground and background geofence listeners, you should unregister your foreground event listener whenever your app is not visible to the user and re-register your app when it becomes visible again. Here's some example code that registers for the visibility event.

```
C#
```

```
Windows.UI.Core.CoreWindow coreWindow;

// This needs to be set before InitializeComponent sets up event
// registration for app visibility
coreWindow = CoreWindow.GetForCurrentThread();
coreWindow.VisibilityChanged += OnVisibilityChanged;
```

When the visibility changes, you can then enable or disable the foreground event handlers as shown here.

```
C#

private void OnVisibilityChanged(CoreWindow sender,
VisibilityChangedEventArgs args)
{
    // NOTE: After the app is no longer visible on the screen and before the
    // app is suspended
    // you might want your app to use toast notification for any geofence
    // activity.
    // By registering for VisibilityChanged the app is notified when the app
    // is no longer visible in the foreground.

    if (args.Visible)
    {
        // register for foreground events
        GeofenceMonitor.Current.GeofenceStateChanged +=
OnGeofenceStateChanged;
        GeofenceMonitor.Current.StatusChanged += OnGeofenceStatusChanged;
    }
    else
    {
        // unregister foreground events (let background capture events)
        GeofenceMonitor.Current.GeofenceStateChanged -=
OnGeofenceStateChanged;
        GeofenceMonitor.Current.StatusChanged -= OnGeofenceStatusChanged;
    }
}
```

Sizing your geofences

While GPS can provide the most accurate location info, geofencing can also use Wi-Fi or other location sensors to determine the user's current position. But using these other methods can affect the size of the geofences you can create. If the accuracy level is low, creating small geofences won't be useful. In general, it is recommended that you do not create a geofence with a radius smaller than 50 meters. Also, geofence background tasks only run periodically on Windows; if you use a small geofence, there's a possibility that you could miss an [Enter](#) or [Exit](#) event entirely.

If your app needs to use a geofence with a small radius, advise users to use your app on a device with a GPS radio to ensure the best performance.

Related topics

- [Set up a geofence](#)
- [Get current location](#)
- [UWP location sample \(geolocation\)](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Map style sheet reference

Article • 06/21/2024

📘 Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#) and [Bing Maps documentation](#).

Microsoft mapping technologies use [map style sheets](#) to define the appearance of maps, including those displayed in a [MapControl](#). A map style sheet is defined using JavaScript Object Notation (JSON) through the [MapStyleSheet.ParseFromJson](#) method.

A style sheet consists of [entries](#) whose properties are overridden to change the final appearance of the map.

For example, the following JSON can be used to make water areas appear in red, water labels appear in green, and land areas appear in blue:

JSON

```
{ "version": "1.*",  
  "settings": { "landColor": "#0000FF" },  
  "elements": { "water": { "fillColor": "#FF0000", "labelColor": "#00FF00" } }  
}
```

Style sheets can be created interactively using the [Map Style Sheet Editor](#) application.

Feedback

Was this page helpful?

👍 Yes

👎 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Porting apps to Windows 10

Article • 10/20/2022

This section describes how to port your existing app to the Universal Windows Platform (UWP) where you can create a single Windows 10 app package that your customers can install onto all types of devices. Your app will benefit from exciting new hardware, great monetization opportunities, a modern API set, adaptive UI controls, and a range of input modalities including mouse/keyboard, touch, and speech.

The Windows Runtime (WinRT) is the technology that lets you build Universal Windows Platform (UWP) apps. You can refer to [What's a Universal Windows Platform \(UWP\) app?](#) for more background on WinRT and UWP apps.

This porting guide explains the differences between your current app's technology and the Universal Windows Platform (UWP). Once the path between technologies is understood, you'll be able to dive into the rest of the Developer Center, which is a comprehensive resource for developing UWP apps. A good way to do that, when you're ready, is to start with [How to develop a Store app](#).

Topic	Description
Move from desktop to UWP	Choose one of several options to bring UWP experiences into your Win32 and .NET desktop applications.
Move from Windows Runtime 8.x to UWP	If you have a Universal 8.1 app—whether it's targeting Windows 8.1, Windows Phone 8.1, or both—then you'll find that your source code and skills will port smoothly to Windows 10 or Windows 11. With Windows, you can create a UWP app, which is a single app package that your customers can install onto every kind of device.
Move from Windows Phone Silverlight to UWP	If you're a developer with a Windows Phone Silverlight app, then you can make great use of your skill set and your source code in the move to Windows 10. With Windows 10 or Windows 11, you can create a UWP app, which is a single app package that your customers can install onto every kind of device.
Convert your web app to a PWA	You can now convert your web app to a Progressive Web App (PWA) will work on any platform, including UWP! The PWA Builder tool will generate the necessary manifest for you . This replaces the Hosted Web Apps (HWA) bridge.

Related topics

- [Move from WPF and Silverlight to WinRT](#) [Move from the web to WinRT](#)

Move from a desktop application to UWP

Article • 10/20/2022

If you have an existing desktop application that was built using the .NET Framework (including WPF and Windows Forms) or C++ Win32 APIs, you have several options for moving to the Universal Windows Platform (UWP) and Windows 10/11.

Package your desktop application in an MSIX package

You can package your desktop application in an MSIX package to get access to many more Windows 10 and Windows 11 features. MSIX is a modern Windows app package format that provides a universal packaging experience for all Windows apps, including UWP, WPF, Windows Forms and Win32 apps. Packaging your desktop Windows apps in MSIX packages gets you access to a robust installation and updating experience, a managed security model with a flexible capability system, support for the Microsoft Store, enterprise management, and many custom distribution models. You can package your application whether you have the source code or if you only have an existing installer file (such as an MSI or App-V installer). After you package your application, you can integrate UWP features such as package extensions and other UWP components.

For more information, see [Building an MSIX package from your code](#) and [Features that require package identity](#).

Use Windows Runtime APIs

You can call many Windows Runtime APIs directly in your WPF, Windows Forms, or C++ Win32 desktop app to integrate modern experiences that light up for Windows 10 users. For example, you can call Windows Runtime APIs to add toast notifications to your desktop app.

For more information, see [Use Windows Runtime APIs in desktop apps](#).

Migrate a .NET Framework app to a UWP app

If your application runs on the .NET Framework, you can migrate it to a UWP app by leveraging .NET Standard 2.0. Move as much code as you can into .NET Standard 2.0

class libraries, and then create a UWP app that references your .NET Standard 2.0 libraries.

Share code in a .NET Standard 2.0 library

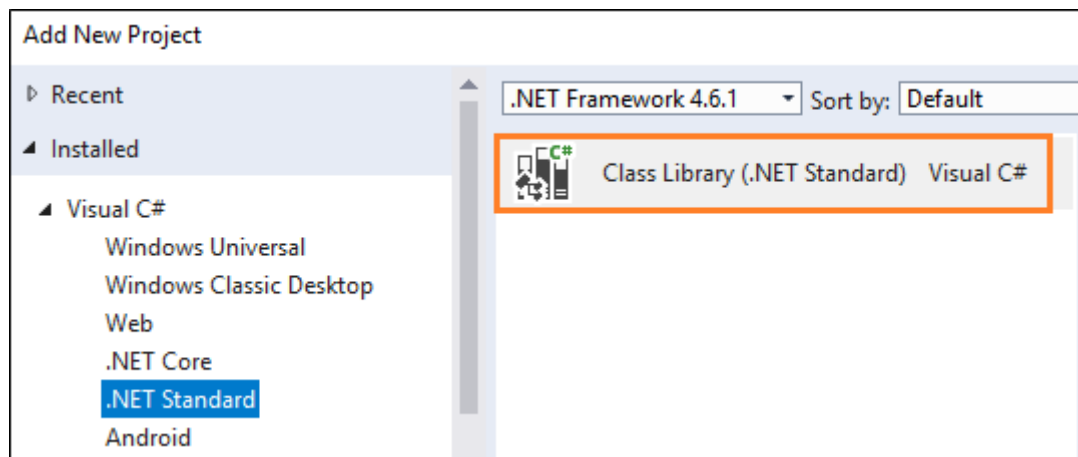
If your application runs on the .NET Framework, place as much code as you can into .NET Standard 2.0 class libraries. As long as your code uses APIs that are defined in the standard, you can reuse it in a UWP app. It's easier than it's ever been to share code in a .NET Standard library because so many more APIs are included in the .NET Standard 2.0.

Here's a video that tells you more about it.

https://www.youtube-nocookie.com/embed/YI4MurjfMn8?list=PLRAdsfhKI4OWx321A_pr-7HhRNk7wOLLY&ecver=1 ↗

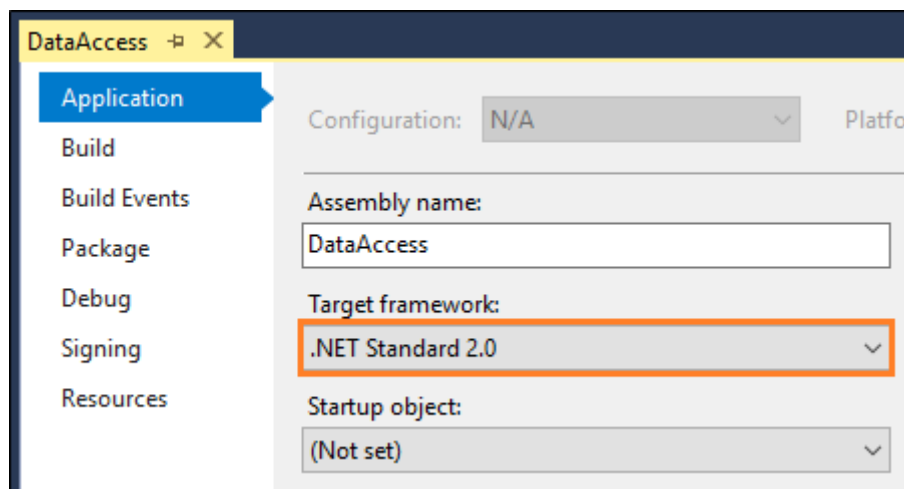
Add .NET Standard libraries

First, add one or more .NET Standard class libraries to your solution.



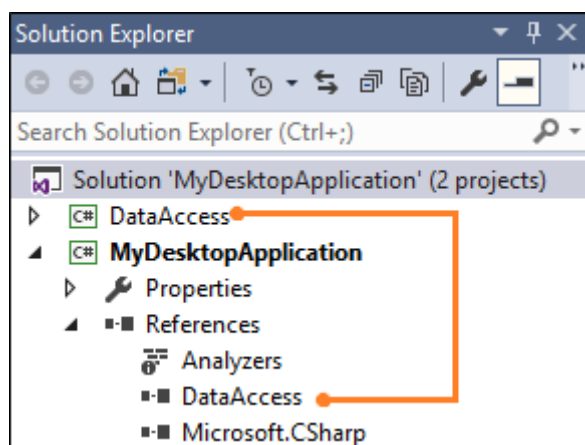
The number of libraries that you add to your solution depends on how you plan to organize your code.

Make sure that each class library targets the **.NET Standard 2.0**.



You can find this setting in the property pages of the class library project.

From your desktop application project, add a reference to the class library project.



Next, use tools to determine how much of your code conforms to the standard. That way, before you move code into the library, you can decide which parts you can reuse, which parts require minimal modification, and which parts will remain application-specific.

Check library and code compatibility

We'll start with Nuget Packages and other dll files that you obtained from a third party.

If your application uses any of them, determine if they are compatible with the .NET Standard 2.0. You can use a Visual Studio extension or a command-line utility to do that.

Use these same tools to analyze your code. Download the tools here ([dotnet-apiport](#)) and then watch this video to learn how to use them.

[https://www.youtube-nocookie.com/embed/rzs_FGPyAIY?](https://www.youtube-nocookie.com/embed/rzs_FGPyAIY?list=PLRAdsfhKI4OWx321A_pr-7HhRNk7wOLLY&ecver=2)

[list=PLRAdsfhKI4OWx321A_pr-7HhRNk7wOLLY&ecver=2](#)

If your code isn't compatible with the standard, consider other ways that you could implement that code. Start by opening the [.NET API Browser](#). You can use that browser

to review the API's that are available in the .NET Standard 2.0. Make sure to scope the list to the .NET Standard 2.0.

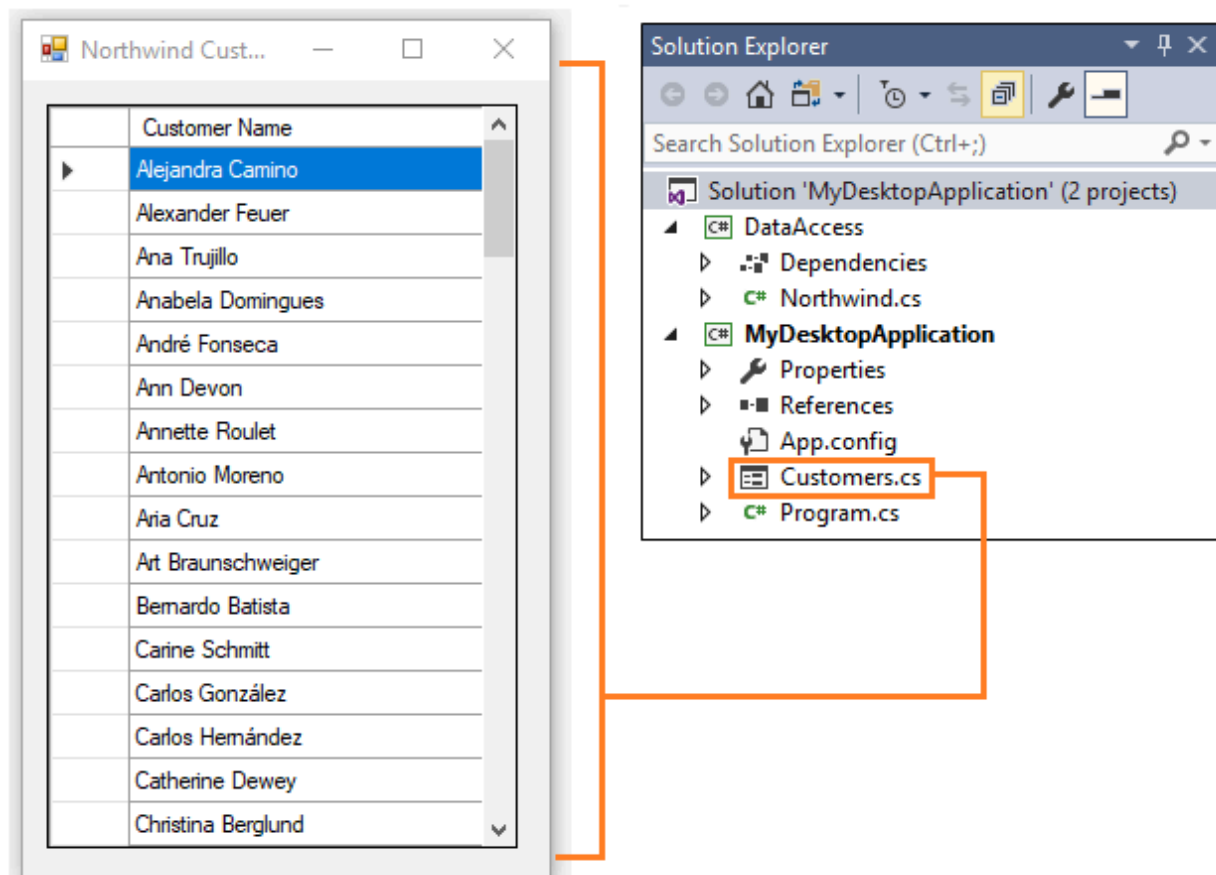


Product	Version
All APIs	2.0
.NET Core	1.6
.NET Framework	1.5
.NET Standard	1.4

Some of your code will be platform-specific and will need to remain in your desktop application project.

Example: Migrating data access code to a .NET Standard 2.0 library

Let's assume that we have a very basic Windows Forms application that shows customers from our Northwind sample database.



The project contains a .NET Standard 2.0 class library with a static class named **Northwind**. If we move this code into the **Northwind** class, it won't compile because it

uses the `SqlConnection`, `SqlCommand`, and `SqlDataReader` classes, and those classes that are not available in the .NET Standard 2.0.

C#

```
public static ArrayList GetCustomerNames()
{
    ArrayList customers = new ArrayList();

    using (SqlConnection conn = new SqlConnection())
    {
        conn.ConnectionString = ...; // Your connection string goes here.

        conn.Open();

        SqlCommand command = new SqlCommand("select ContactName from
customers order by ContactName asc", conn);

        using (SqlDataReader reader = command.ExecuteReader())
        {
            while (reader.Read())
            {
                customers.Add(reader[0]);
            }
        }
    }

    return customers;
}
```

We can use the [.NET API Browser](#) to find an alternative though. The `DbConnection`, `DbCommand`, and `DbDataReader` classes are all available in the .NET Standard 2.0 so we can use them instead.

This revised version uses those classes to get a list of customers, but to create a `DbConnection` class, we'll need to pass in a factory object that we create in the client application.

C#

```
public static ArrayList GetCustomerNames(DbProviderFactory factory)
{
    ArrayList customers = new ArrayList();

    using (DbConnection conn = factory.CreateConnection())
    {
        conn.ConnectionString = ...; // Your connection string goes here.

        conn.Open();
```

```

        DbCommand command = factory.CreateCommand();
        command.Connection = conn;
        command.CommandText = "select ContactName from customers order by
ContactName asc";

        using (DbDataReader reader = command.ExecuteReader())
        {
            while (reader.Read())
            {
                customers.Add(reader[0]);
            }
        }

        return customers;
    }
}

```

In the code-behind page of the Windows Form, we can just create factory instance and pass it into our method.

C#

```

public partial class Customers : Form
{
    public Customers()
    {
        InitializeComponent();

        dataGridView1.Rows.Clear();

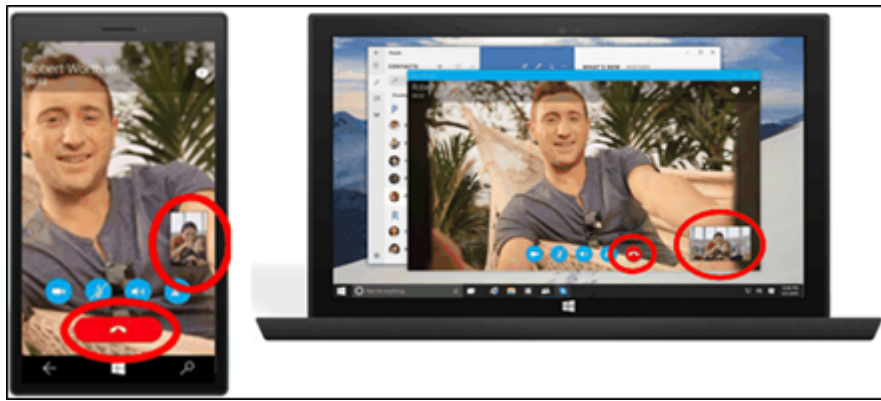
        SqlClientFactory factory = SqlClientFactory.Instance;

        foreach (string customer in Northwind.GetCustomerNames(factory))
        {
            dataGridView1.Rows.Add(customer);
        }
    }
}

```

Create a UWP app

Now you're ready to add a UWP app to your solution.



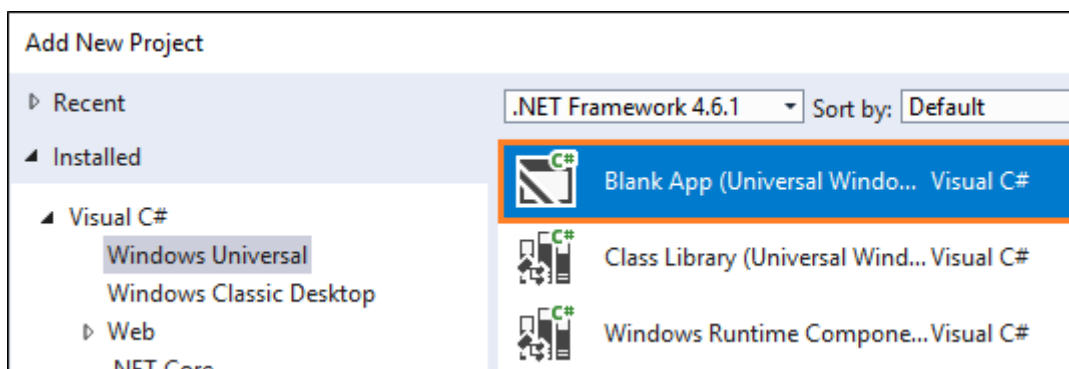
You'll still have to design UI pages in XAML and write any device or platform-specific code, but when you are done, you'll be able to reach the full breadth of Windows 10 and Windows 11 devices and your app pages will have a modern feel that adapts well to different screen sizes and resolutions.

Your app will respond to input mechanisms other than just a keyboard and mouse, and features and settings will be intuitive across devices. This means that users learn how to do things one time, and then it works in a very familiar way no matter the device.

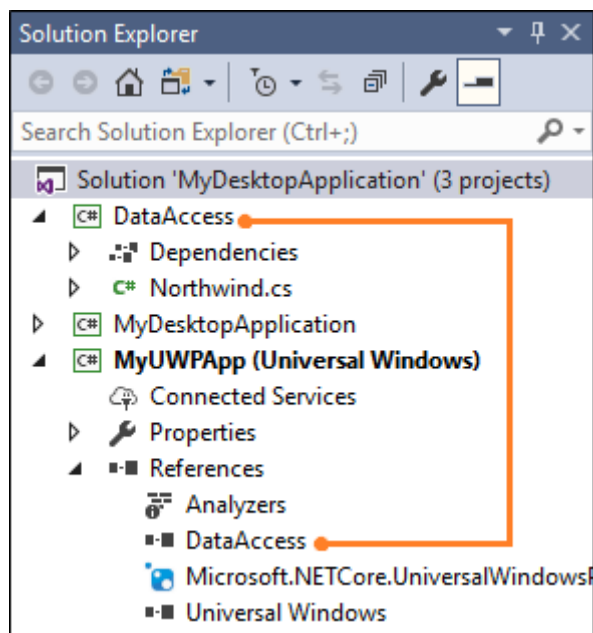
These are just a few of the goodies that come with UWP. To learn more, see [Build great experiences with Windows](#).

Add a UWP project

First, add a UWP project to your solution.

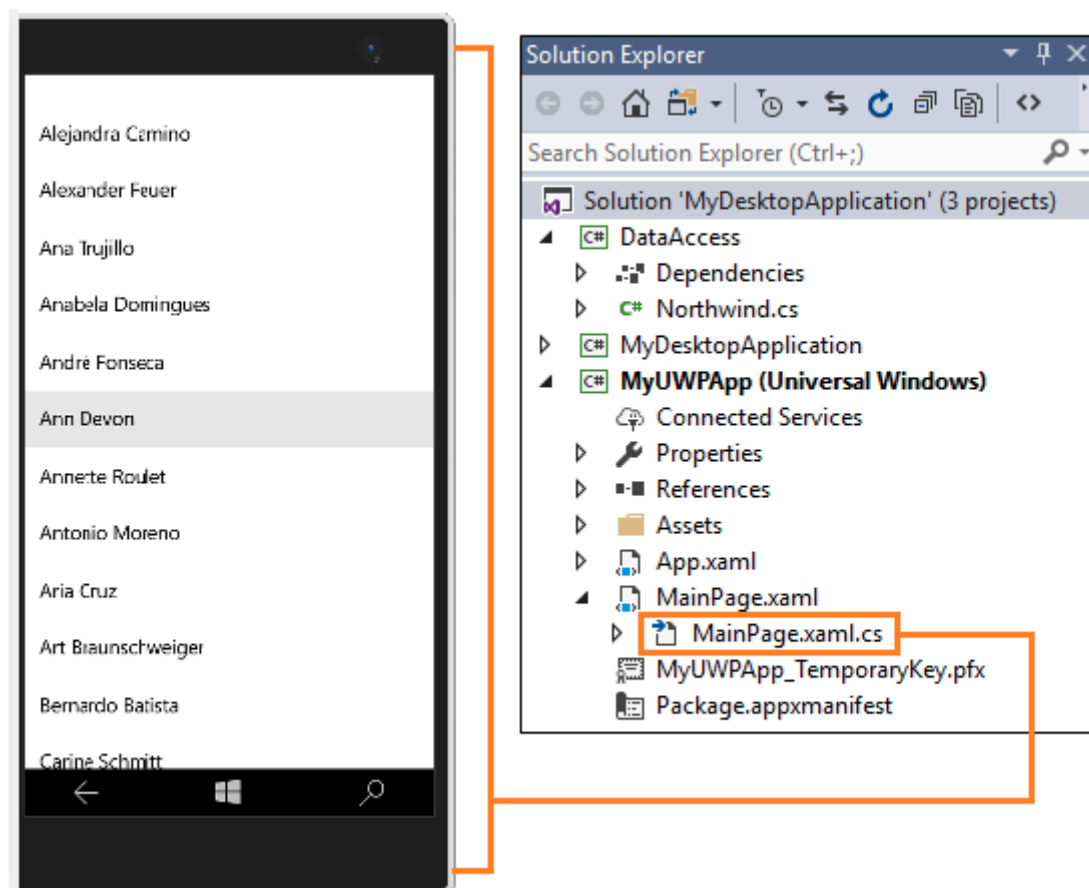


Then, from your UWP project, add a reference the .NET Standard 2.0 library project.



Build your pages

Add XAML pages and call the code in your .NET Standard 2.0 library.



XML

```
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
    <StackPanel x:Name="customerStackPanel">
        <ListView x:Name="customerList"/>
    </StackPanel>
</Grid>
```

```
</StackPanel>
</Grid>
```

C#

```
public sealed partial class MainPage : Page
{
    public MainPage()
    {
        this.InitializeComponent();

        SqlClientFactory factory = SqlClientFactory.Instance;

        customerList.ItemsSource = Northwind.GetCustomerNames(factory);
    }
}
```

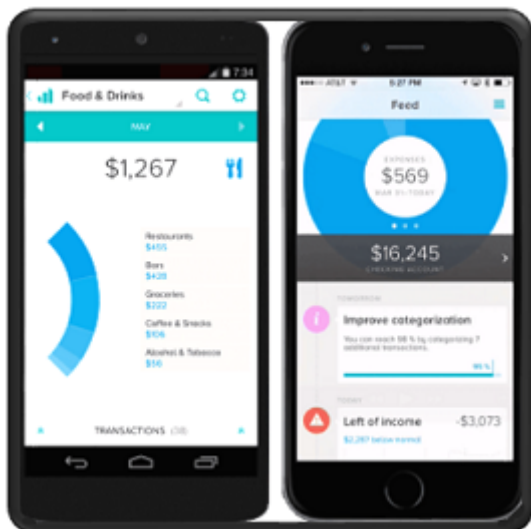
To get started with UWP, see [What's a UWP app](#).

Reach iOS and Android devices

You can reach Android and iOS devices by adding Xamarin projects.

⚠ Note

For new cross-platform projects, please consider using .NET MAUI.



These projects let you use C# to build Android and iOS apps with full access to platform-specific and device-specific APIs. These apps leverage platform-specific hardware acceleration, and are compiled for native performance.

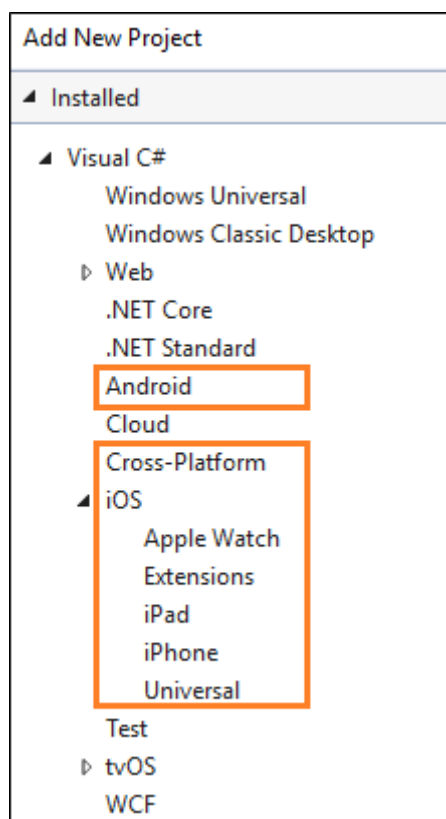
They have access to the full spectrum of functionality exposed by the underlying platform and device, including platform-specific capabilities like iBeacons and Android Fragments and you'll use standard native user interface controls to build UIs that look and feel the way that users expect them to.

Just like UWP, the cost to add an Android or iOS app is lower because you can reuse business logic in a .NET Standard 2.0 class library. You'll have to design your UI pages in XAML and write any device or platform-specific code.

Add a Xamarin project

First, add an **Android**, **iOS**, or **Cross-Platform** project to your solution.

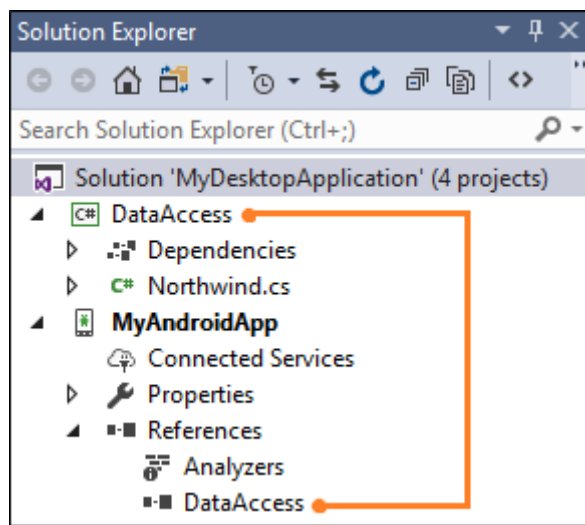
You can find these templates in the **Add New Project** dialog box under the **Visual C#** group.



ⓘ Note

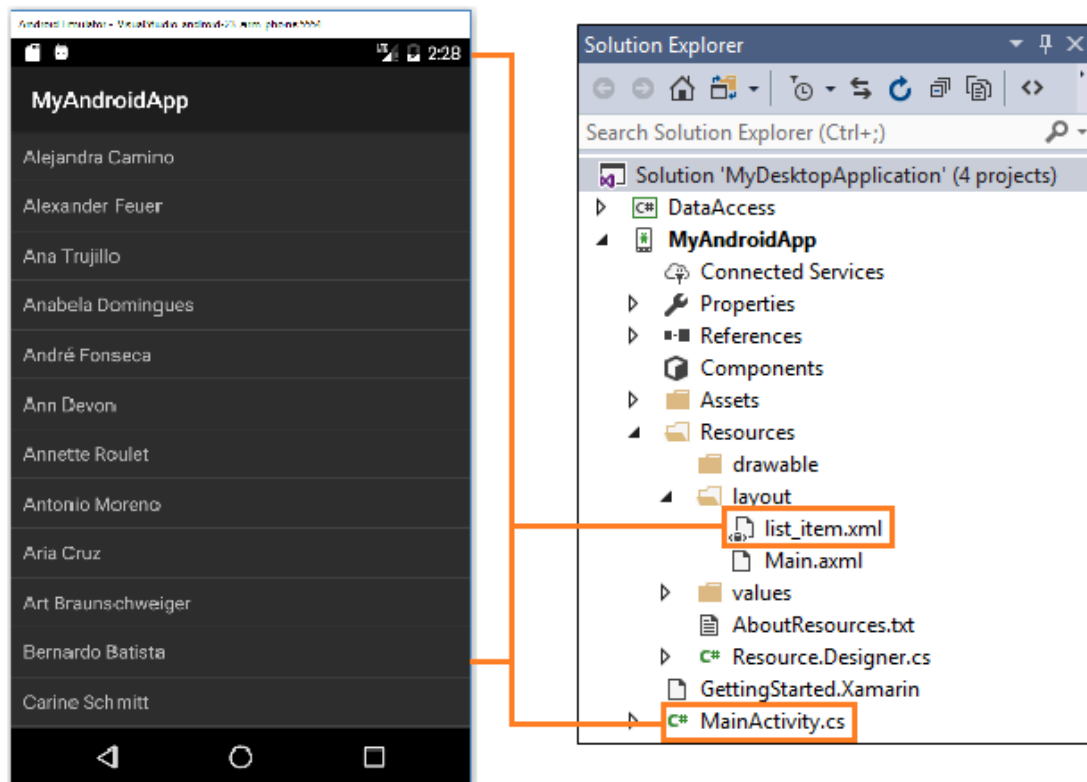
Cross-platform projects are great for apps with little platform-specific functionality. You can use them to build one native XAML-based UI that runs on iOS, Android, and Windows. Learn more [here](#).

Then, from your Android, iOS, or cross-platform project, add a reference the class library project.



Build your pages

Our example shows a list of customers in an Android app.



XML

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:padding="10dp" android:textSize="16sp"
    android:id="@android:id/list">
</TextView>
```

C#

```
[Activity(Label = "MyAndroidApp", MainLauncher = true)]
public class MainActivity : ListActivity
{
    protected override void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);

        SqlClientFactory factory = SqlClientFactory.Instance;

        var customers =
            (string[])Northwind.GetCustomerNames(factory).ToArray(typeof(string));

        ListAdapter = new ArrayAdapter<string>(this,
            Resource.Layout.list_item, customers);
    }
}
```

To get started with Android, iOS, and cross-platform projects, see the [Xamarin developer portal](#).

Next steps

Find answers to your questions

Have questions? Ask us on Stack Overflow. Our team monitors these [tags](#). You can also ask us [here](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)